theme_project.R

norazhan

2026-10-19

```r
# theme_project.R
# Colors, fonts, and chart style for "Who Builds Federal AI?" (PPOL 5202, Fall 2026)
# How to use: put source("theme_project.R") at the top of every figure file.
# Packages needed: ggplot2 and systemfonts (ragg is used too, if installed)

library(ggplot2)
```

```r
# Each color is named by its job, not by how it looks
proj_colors <- c(
  ink       = "#1F2933",  # dark text: titles, labels, reference lines
  ink_soft  = "#5B6670",  # lighter text: subtitles, axis text, notes
  paper     = "#FFFFFF",  # background
  context   = "#B9C0C8",  # gray for anything that is not the main point
  grid      = "#E5E8EB",  # very light gray for gridlines
  highlight = "#0A908A",  # teal for the one bar or line the title is about
  missing   = "#EFEDE9"   # pale gray for "Not reported"
)
```

```r
# Rule: a color means the same thing in every chart of the project.
# Only three colors, at medium lightness:
#   blue  = AI the government builds itself (in-house)
#   coral = AI bought from a vendor
#   teal  = the AI itself (stage, counts of use cases, the highlight)

# Who built the AI: coral = bought from a vendor, blue = built in-house
pal_build <- c(
  "Vendor"              = "#CF857A",
  "Vendor and in-house" = "#D3CEC6",
  "In-house"            = "#4F759D"
)

# Stage: light to dark teal, darker = further along
pal_stage <- c(
  "Pre-deployment" = "#C1E3E1",
  "Pilot"          = "#65B9B5",
  "Deployed"       = "#017A74",
  "Retired"        = "#B9C0C8"   # gray: no longer in use
)

# Type of AI: no rainbow; newer types in teal, the rest in gray
pal_ai_type <- c(
  "Generative AI"   = "#0A908A",
  "Agentic AI"      = "#0A908A",
```

```r
  "Classical ML"    = "#B9C0C8",
  "NLP"             = "#B9C0C8",
  "Computer Vision" = "#B9C0C8"
)

# Counts of AI use cases (e.g., a heatmap): light teal = few, dark teal = many
pal_volume <- c(low = "#E0F1F0", high = "#017A74")
```

```r
# One font, with backups. R uses the first one that is installed:
# Public Sans (U.S. government websites) -> Helvetica Neue (Mac) -> Arial (Windows) -> sans
font_stack <- c("Public Sans", "Helvetica Neue", "Arial", "sans")

# Go down the list and return the first font this computer has
pick_font <- function(stack) {
  installed <- systemfonts::system_fonts()$family
  for (f in stack) {
    if (f %in% installed) return(f)
  }
  "sans"
}

proj_font <- pick_font(font_stack)

# Text sizes in points. geom_text() measures size in millimeters,
# so those sizes are divided by .pt to convert
size_title      <- 15
size_subtitle   <- 11
size_axis       <- 10
size_label      <- 10 / .pt   # labels on or next to bars and lines
size_annotation <- 9 / .pt    # short notes inside the chart
size_caption    <- 8.5

# Make every text label use the project font and dark ink unless told otherwise
update_geom_defaults("text",  list(family = proj_font, colour = proj_colors[["ink"]],
                                   size = size_label))
update_geom_defaults("label", list(family = proj_font, colour = proj_colors[["ink"]],
                                   size = size_label))

# In Quarto, draw figures with ragg so the fonts above actually show up
if (requireNamespace("ragg", quietly = TRUE) && requireNamespace("knitr", quietly = TRUE)) {
  knitr::opts_chunk$set(dev = "ragg_png")
}
```

```r
# The theme itself. grid = which gridlines to keep: "x", "y", "both", or "none"
theme_project <- function(base_size = 11, grid = "x") {
  ink      <- proj_colors[["ink"]]
  ink_soft <- proj_colors[["ink_soft"]]
  grid_col <- proj_colors[["grid"]]

  t <- theme_minimal(base_size = base_size, base_family = proj_font) +
    theme(
      # Text: title biggest, then subtitle, then axis text, then the note at the bottom
      plot.title = element_text(size = size_title, face = "bold", colour = ink,
                                margin = margin(b = 4)),
```

```
      plot.subtitle = element_text(size = size_subtitle, colour = ink_soft,
                                   margin = margin(b = 12)),
      plot.caption = element_text(size = size_caption, colour = ink_soft,
                                  hjust = 0, lineheight = 1.15,
                                  margin = margin(t = 10)),
      plot.title.position = "plot",     # line the title up with the left edge
      plot.caption.position = "plot",
      axis.text = element_text(size = size_axis, colour = ink_soft),
      axis.title = element_text(size = size_axis, colour = ink_soft),
      strip.text = element_text(size = size_axis, face = "bold", colour = ink,
                                hjust = 0),
      legend.text = element_text(size = size_axis, colour = ink),
      legend.title = element_text(size = size_axis, colour = ink_soft),
      legend.position = "top",
      legend.justification = "left",
      # Background: white, with thin light gridlines only
      panel.grid.major = element_line(colour = grid_col, linewidth = 0.4),
      panel.grid.minor = element_blank(),
      plot.background = element_rect(fill = proj_colors[["paper"]], colour = NA),
      plot.margin = margin(12, 16, 10, 12)
    )

  if (grid == "x")    t <- t + theme(panel.grid.major.y = element_blank())
  if (grid == "y")    t <- t + theme(panel.grid.major.x = element_blank())
  if (grid == "none") t <- t + theme(panel.grid.major = element_blank())
  t
}
```

```
# e.g. scale_fill_stage() instead of typing the stage colors by hand.
# Missing values get the "missing" gray.
scale_fill_build <- function(...) {
  scale_fill_manual(values = pal_build, na.value = proj_colors[["missing"]], ...)
}

scale_fill_stage <- function(...) {
  scale_fill_manual(values = pal_stage, na.value = proj_colors[["missing"]], ...)
}

scale_fill_volume <- function(...) {
  scale_fill_gradient(low = pal_volume[["low"]], high = pal_volume[["high"]],
                      na.value = proj_colors[["missing"]], ...)
}
```